Föreläsning 5

Rekursion

Föreläsning 5

- Algoritm
- Rekursion
- Rekursionsträd
- Funktionsanrop på stacken
- Binär sökning
- Problemlösning (möjliga vägar)

Algoritm

En algoritm är ett begränsat antal instruktioner/steg för att lösa en uppgift, som från givna indata med säkerhet leder till korrekta utdata.

Precision - varje steg är exakt bestämt

Determinism -resultatet av varje steg är entydigt

Ändlig - når målet efter ett ändligt antal steg

Exempel

Problem: Hitta det största av tre heltal

Algoritm:

1. Kalla talen a, b och c
2. Sätt x = a
3. Om b > x sätt x = b
4. Om c > x sätt x = c
5. Svar: x

Frågor

- Terminerar algoritmen
- Fungerar den för alla giltiga indata (gränsvärden)
- Producerar den korrekt resultat
- Är den tillräckligt effektiv, går den att effektivisera?

Rekursion

- Rekursion är en mycket mäktig problemlösnings-strategi
- Det är ibland det enklaste sättet att lösa ett problem och kräver då mycket mindre kod än alternativet (iteration)
- Däremot är det inte säkert att lösningen blir effektiv och den kan ofta använda mer minne
- För den ovane känns rekursion krångligt men när man fått grepp om tekniken är den oumbärlig

Men vad är rekursion?
För att förstå rekursion måste man först förstå rekursion…
Vi menar här främst att en metod anropar sig själv för att lösa ett problem

Rekursivt definierad talföljd

Innan vi tittar på rekursion för problemlösning värmer vi upp med en rekursivt definierad talföljd

Fibonacci-följden:

$f_n = f_{n-1} + f_{n-2}$, n=3,4,5,…

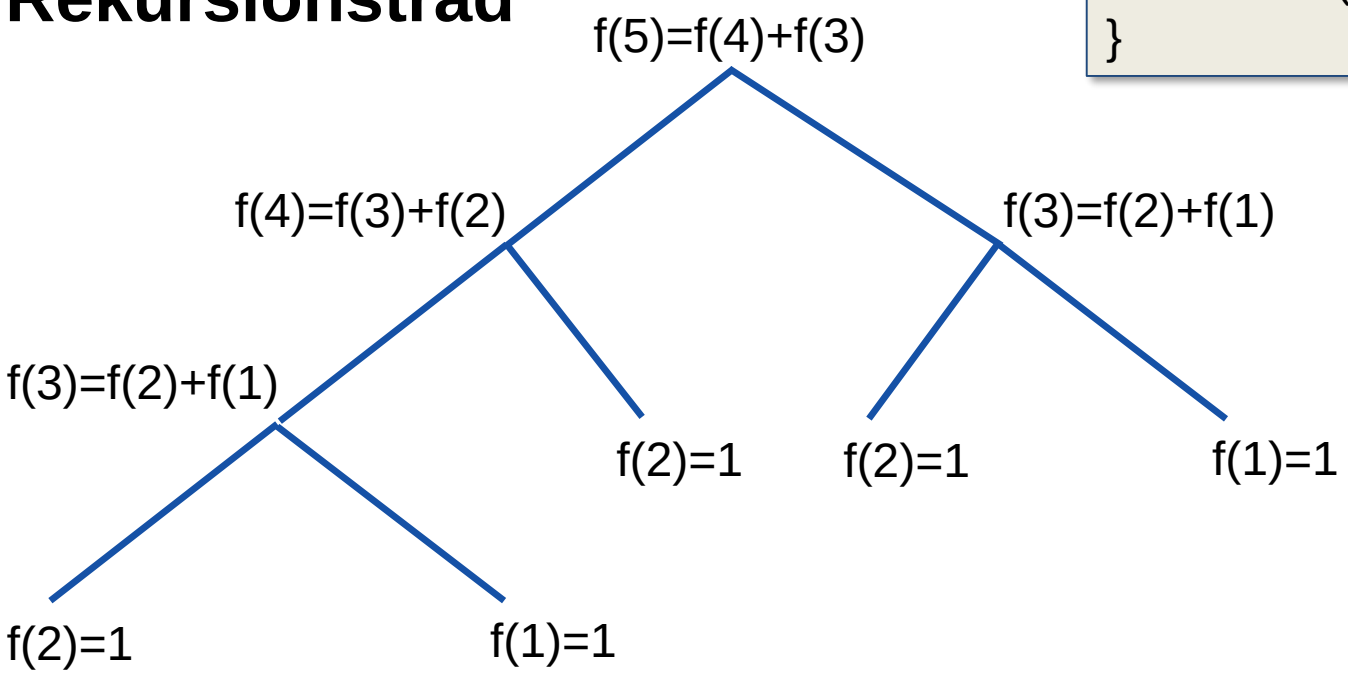
$f_1 = f_2 = 1$ (termineringsvillkor – viktigt!)

1, 1, 2, 3, 5, 8, 13, 21,…

```
static int fib(int n){
   if(n==1||n==2) return 1;
   return fib(n-1)+fib(n-2);
}
```

Rekursionsträd

```
static int fib(int n){
   if(n==1||n==2) return 1;
   return fib(n-1)+fib(n-2);
}
```



Observera att vi får ”räkna” ut f(3) två gånger. Om n ökar från t.ex från 100 till 101 måste vi alltså räkna ut f(100) (som förut) men nu också f(99). Vi dubblar alltså antalet beräkningar när n ökar med 1: $O(2^n)$.

Denna mycket snygga lösning kodmässigt är alltså mycket in-effektiv. Vi kommer senare att fixa detta med dynamiskt programmering så att den snygga koden också blir effektiv (O(n)).

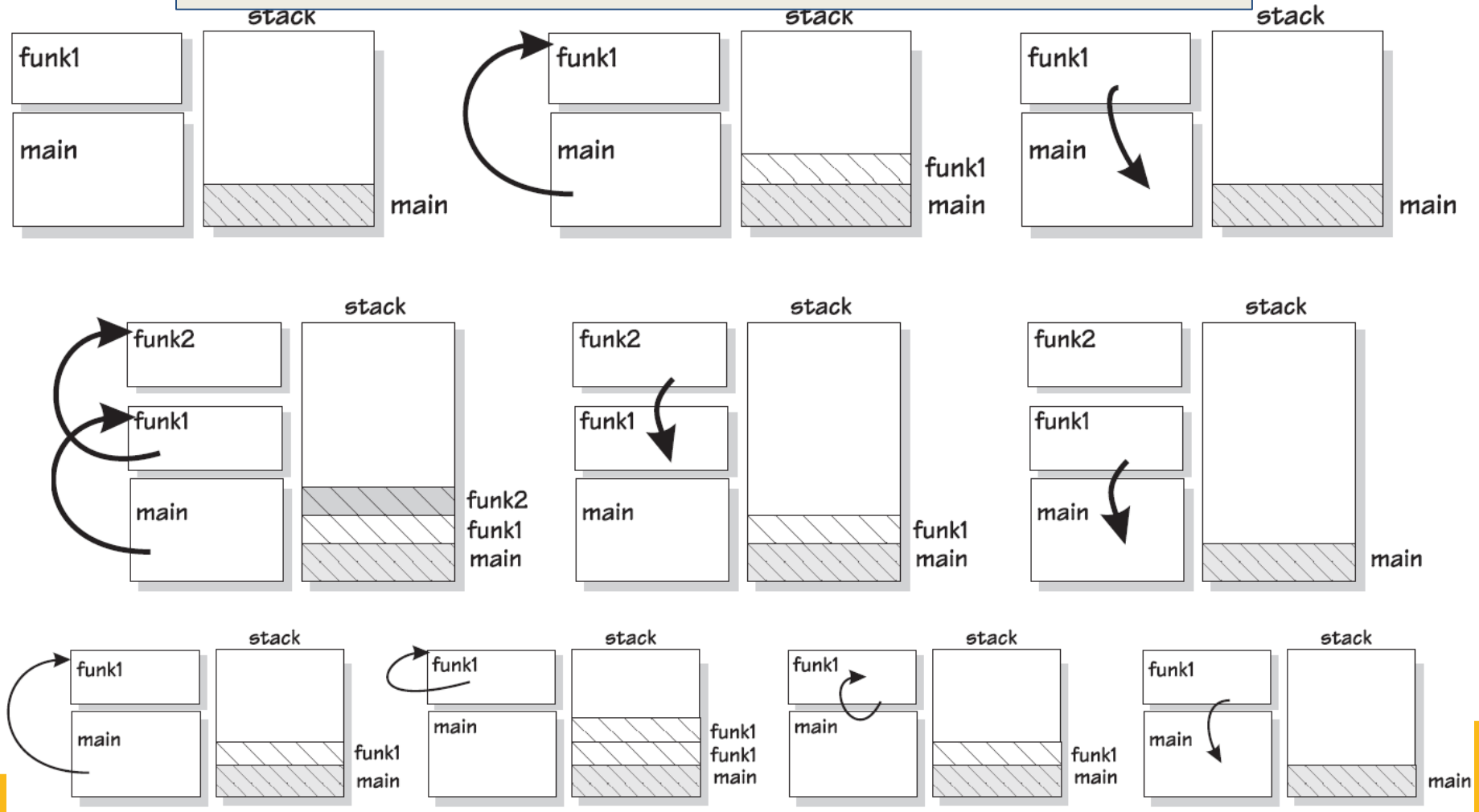
Minneshantering i JVM

JVM organiserar datat till en körande java-applikation i följande områden: stackar (en per tråd), en heap, och ett metodområde (och ett register).

- I stacken sparar tråden lokala variabler, parametrar, och metodanrop. Endast primitiva typer och referenser finns här – inga objekt
- Det finns bara en heap. Här bor alla objekt som skapas.
- Metodområdet består av alla klass-variabler som programmet använder och delas av alla trådar. Innehåller precis som stacken endast primitiva datatyper och referenser.

Stacken vid funktionsanrop

När en funktion anropas så skapas utrymme på *stacken*
för de *lokala variablerna*, *parametrarna* och *återhoppsadressen*
Detta gör att en rekursiv lösning ofta kräver mer minne än en iterativ.



```
static int fib(int n){
   if(n==1||n==2) return 1;
   return fib(n-1)+fib(n-2);
}
```

Rekursivt-iterativt

Det kan bevisas att alla problem som kan lösas rekursivt också kan lösas iterativt.

```
static int fib(int nFinal)
{
   int fn=1,fnMinus1=1,fnMinus2=1;
   for(int n=3;n<=nFinal;n++)
   {
      fn=fnMinus1+fnMinus2;
      fnMinus2=fnMinus1;
      fnMinus1=fn;
   }
   return fn;
}//O(n)
```

Varje värde beräknas en gång!

Ännu bättre?:

$$f_n = \frac{\left(\frac{1+\sqrt{5}}{2}\right)^n - \left(\frac{1-\sqrt{5}}{2}\right)^n}{\sqrt{5}}$$

Vi observerar att den iterativa också blir minnes-effektivare då den kan återanvända samma variabler medans den rekursiva behöver nya kopior av alla variabler för varje anrop

Svansrekursion

```
static int fib(int n){
   if(n==1||n==2) return 1;
   return fib(n-1)+fib(n-2);
}
```

Endast ett rekursivt anrop och detta är det sista som sker i funktionen.

Kompilatorer kan effektivisera svansrekursion. Vissa kompilatorer kan skriva om svansrekursion till iteration och därmed spara den overhead som varje funktionsanrop har.

```
static int fib(int n){
   if(n==1||n==2) return 1;
   return fn(1,1,3,n);
}

static int fn(int fnMinus1,int fnMinus2,int n,int nFinal){
   if(n==nFinal) return fnMinus1+fnMinus2;
   return fn(fnMinus1+fnMinus2,fnMinus1,n+1,nFinal);
}
```

Att omformulera till svansrekursivt görs främst i språk som saknar iteration. När vi löser problem rekursivt siktar vi på rena lättlästa lösningar som den i hörnet och effektiviserar dem med dynamisk programmering (alternativt skriver om dem med iteration).

Fakultet

Definition: n! = 1·2·…·(n-1)·n

Ex: 5! = 1·2·3·4·5

```
int fak(int nFinal){
    int fakn=1;
    for(int n=2;n<=nFinal;n++)
        fakn*=n;
    return fakn;
}//O(n)
```

Den rekursiva lösningen får vi genom att observera att

5! = 5·4! eller n!=n·(n-1)!

```
int fak(int n){
    if(n==0) return 1;
    return n*fak(n-1);
}//O(n)
```

Denna är inte svansrekursiv pga multiplikationen som kvarstår

```
public static int fak(int n){
    return fak(n,1,1);
}
private static int fak(int n, int result, int factor) {
    if(factor==n) return result*factor;
    return fak2(n,result*factor, factor+1);
}//O(n)
```

Denna är förvisso svansrekursiv men inte alls lika snygg. När man samlar ihop resultatet i en variabel som man sedan returnerar svarar varje funktionsanrop inte på ett delproblem.

Linjär sökning i array

Vi söker efter ett objekt i en array med n element genom att börja med första elementet och jobba oss framåt

I genomsnitt krävs (1+n)/2 jämförelser om objektet finns.

Om objektet inte finns krävs alltid n jämförelser

Linjär sökning är O(n)

Algoritm för rekursiv linjär sökning

`Om` arrayen är tom return -1

`annars om` första elementet matchar returnera första elementets index

`annars` returnera resultatet av en sökning av arrayen exklusive första elementet

Kod

```
//Wrapper
public static int linearsearch(Object[] array, Object target){
   return linearSearch(array,target, 0);
}

private static int linearSearch(Object[] array, Object target, int position){
   if(position== array.length) return -1;
   if(target.equals(array[position])) return position;
   return linearSearch(array, target, position+1);
}
```

Binär sökning

Bygger på att vi letar i ett sorterat material.

Algoritm:

```
if
        the array is empty return –1
else if
        the middle element matches the target
        return the subscript of the middle element
else if
        the target is less than the middle element
        search the array elements before the  middle
        element and return the result
else
        search the array elements after the middle
        element  and return the result
```

Binär sökning

```
public static int binarySearch(Object[] items, Comparable target) {
    return binarySearch(items, target, 0, items.length - 1);
}

private static int binarySearch(Object[] items, Comparable target, int first, int last) {
    if (first > last) return -1;
    int middle = (first + last) / 2;
    int compResult = target.compareTo(items[middle]);
    if (compResult == 0) return middle;
    if (compResult < 0) return binarySearch(items, target, first, middle - 1);
    return binarySearch(items, target, middle + 1, last);
}
```

Analys av binär sökning

Låt oss analysera värsta fallet då objektet vi söker saknas. Antalet element är storleken n. T(n) är antalet enkla satser.

T(1) = 1 (egentligen 6 men ointressant för ordo)

T(n) = 1 + T(n/2) (halverar materialet vid varje steg, egentligen 7 + T(n/2))

Detta ger:	Jmf:
T(1) = 1	$1+\log_2 1 = 1$
T(2) = 1 + T(1) = 2	$1+\log_2 2 = 2$
T(4) = 1 + T(2) = 3	$1+\log_2 4 = 3$
T(8) = 1 + T(4) = 4	$1+\log_2 8 = 4$

En dubbling av sökmaterialet ger en ökning med 1

T(n) = 1 + log(n) = O(log(n))

$\log(2^n)=n\log(2)$

$\log(2^{n+1})=(n+1)\log(2)$

Med T(1)=6 och T(n)=7+T(n/2) får vi T(n)=6+7log(n)

Största gemensamma delaren

Greatest common divisor: gcd(78,21)=3

Fås enklast med Euklides algoritm:

78 = 3 · **21** + 15 ger gcd(21,15)

21 = 1 · **15** + 6 ger gcd(15,6)

15 = 2 · **6** + 3 ger gcd(6,3)

6 = 2 · **3** + 0 ger gcd(3,0)

och då är svaret 3!

Algoritm gcd(a,b)

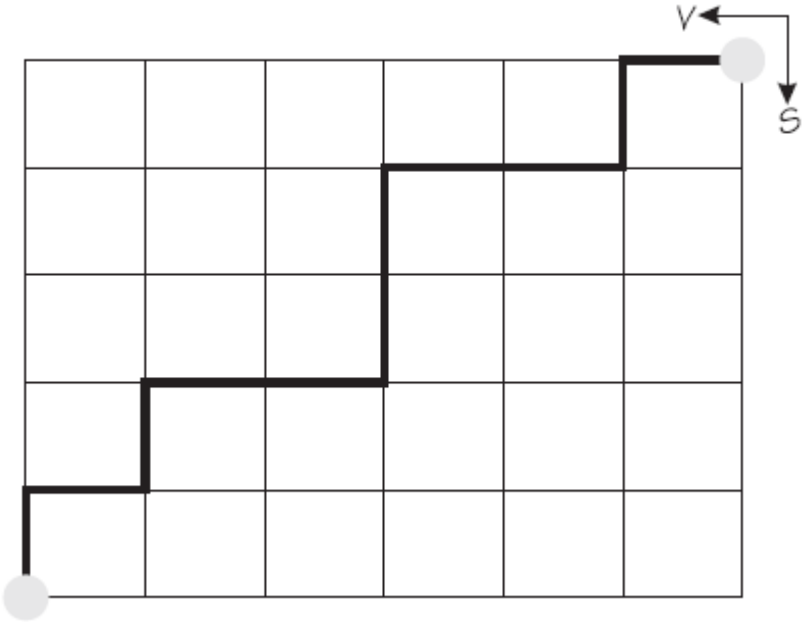
Om b ==0 return a

annars return gcd(b,a%b)

Skriv kod och testa!

Antal möjliga vägar

Hur många unika vägar finns det från övre högra hörnet till nedre vänstra hörnet om vi bara får gå väst och syd?

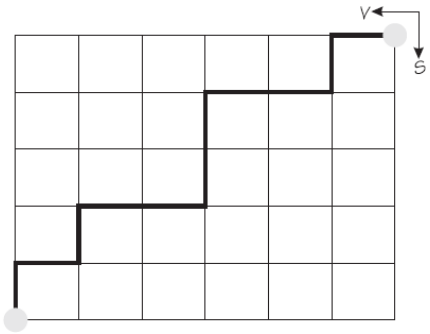


Lösning

- Vi löser problemet genom att gå alla vägar och räkna hur många det blir.
- Låt m vara antal rader och n vara antal kolumner
- Vid varje vägval kan vi då välja att gå väst och därmed minska n med ett eller gå syd och minska m med ett
- När m och n är noll är vi framme och har därmed hittat en väg

n = 6

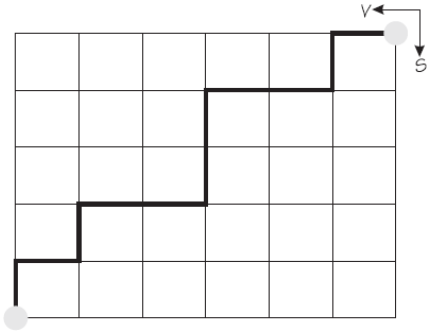
m = 5



Algoritm

n = 6

m = 5



<u>antalVägar(m,n)</u>

Om m = 0 och n = = 0 returnera 1

annars

antal = 0

om m > 0 antal = antalVägar(m-1,n)

om n > 0 antal = antal + antalVägar(m,n-1)

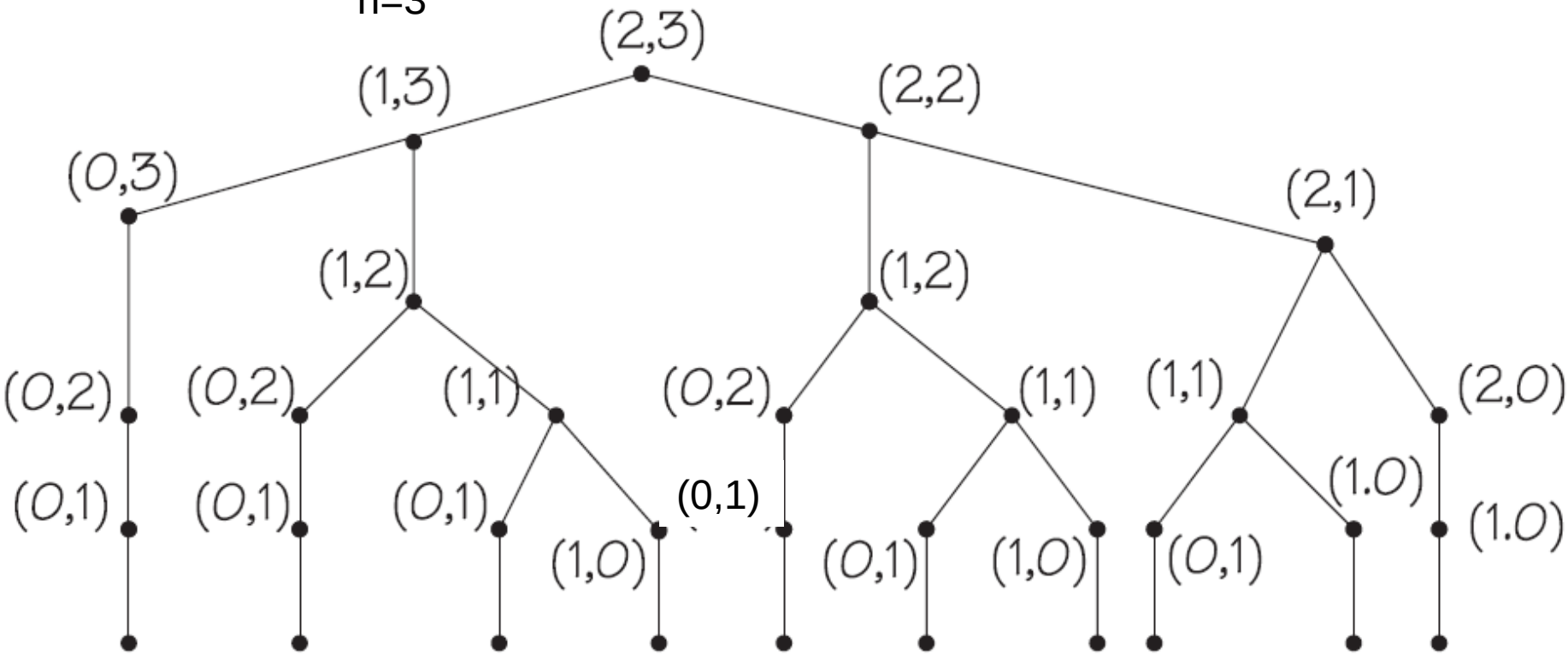
returnera antal

Skriv kod nu

```
static int nrPaths(int m, int n){
        if(n==0 && m==0) return 1;
        int nr = 0;
        if(m>0) nr = nrPaths(m-1,n);
        if(n>0) nr += nrPaths(m,n-1);
        return nr;
}
```

Rekursionsträd

```
static int nrPaths(int m, int n){
        if(n==0 && m==0) return 1;
        int nr = 0;
        if(m>0) nr = nrPaths(m-1,n);
        if(n>0) nr += nrPaths(m,n-1);
        return nr;
}
```



djupet först!